

How Far Off the Recorded Path Can a Gaussian-Splat Driving Simulator Be Trusted?

CS 566 Project Proposal (one document per team) · Kyle Sha (ID 9085525476) · Allen Xia (ID 9085517234) · 29 Sep 2026

Main papers: [1] **Difix3D+** (CVPR 2025 Oral), *re-implemented* · [2] **FaithFusion** (CVPR 2026), *compared* · [3] **HUGSIM** (TPAMI 2026), *simulator for evaluation*

1. Problem Statement

Driving software is increasingly evaluated in simulators reconstructed from real driving logs: the recorded video becomes a **3D Gaussian splat** [13], a scene of millions of small, semi-transparent coloured ellipsoids that can be rendered from any camera position, so the simulated vehicle can leave the recorded trajectory. However, **a reconstruction is reliable only near that trajectory**; after a lane change or with a higher camera (e.g., a truck), rendered images degrade, and a driving system tested on them may behave differently than in reality. This project addresses two questions: **how far from the recorded trajectory a splat-based simulator remains trustworthy, and whether current generative repair methods ("fixers") restore trustworthiness or merely improve visual realism**. We define **trust** as the agreement between the outputs of a lane detector and a vehicle detector on a rendered image and on the **real** image captured at the same pose.

2. Significance and Motivation

Simulation is a principal means of validating driving software at scale; if off-trajectory renders are unreliable, a simulator may accept unsafe behaviour or reject safe behaviour. Both authors are members of Wisconsin Autonomous, a student autonomous-vehicle team, where we develop and test driving software in conventional simulators (MATLAB/Simulink, RoadRunner, CARLA). Their hand-built 3D assets differ visibly from real camera imagery, which limits how far results transfer to the vehicle. Reconstruction-based simulators promise to close this gap, but only if renders stay faithful off the recorded trajectory, and a generative fixer can make an image more convincing while making it less accurate. Understanding this trade-off motivates the project, which draws on camera geometry, 3D reconstruction, image restoration, and CNN-based perception.

3. State of the Art

- **Splat-based simulators:** HUGSIM [3] executes driving policies in reconstructed scenes; the EUVS benchmark [7] shows that current methods degrade under large viewpoint changes.
- **Diffusion-based fixers:** Difix3D+ [1] repairs off-trajectory renders with a single-step diffusion model and adds the repaired views back as training images for the 3D scene, reporting roughly half the FID (a realism score; lower is better) of prior methods. NVIDIA Fixer [4] applies the same approach to driving data, and FaithFusion [2] restricts which pixels a fixer may overwrite, using a per-pixel map of how well the reconstruction already supports each pixel.
- **Evaluation practice:** these methods are evaluated primarily with image-realism metrics (FID, LPIPS) or with shifted cameras for which no real image exists (PRG-Fusion [6]). Some studies do use offset analysis or real off-trajectory images [9, 10, 11], but mainly with image-similarity metrics; we found few comparisons of fixers against real off-trajectory images that also count invented content.

4. Proposed Approach

- **Re-implementation:** we re-implement Difix3D+ [1], comprising the fixer's training procedure and its progressive update, which adds repaired views back as training images for the 3D scene. As in the paper, the fixer is fine-tuned from the pretrained SD-Turbo diffusion model; we write the training code ourselves from the paper, validate it against NVIDIA's official code and weights, and train it on driving data. A preliminary version already runs (§5).
- **What we add around the re-implementation** (applying and combining existing components rather than proposing a new method): (a) an **evaluation against real images**, using Para-Lane [8], which scans the same roads in three parallel lanes (approximately 3 m and 6–7.7 m apart), and MV2 [9], in which a car and a scooter record the same road at different camera heights; training pairs are built from neighbouring lanes, each pairing a render moved into lane B with the real lane-B image (cf. ReCamDriving [12]), and are used to train our Difix and the smaller fixer; (b) **hallucination accounting**, measuring invented or erased lane markings and spurious vehicles in addition to image similarity; (c) a **smaller fixer** trained from scratch on the same pairs (a standard 15M-parameter U-Net), to see how much of Difix's behaviour a much smaller model retains; (d) the **road-surface prior** from ViSE [15], re-implemented because no code is released, which fits a smooth road surface to LiDAR and involves no image generation; and (e) a **cross-dataset check** on US city scenes (EUVS benchmark [7], built on the MARS dataset) and Indian roads (MV2 [9]).

5. Preliminary Observations and Open Questions

We are not proposing a new method. A preliminary run (September 27–28, two held-out test roads) gave these observations, to be checked on more data:

- Rendered images lose lane detail quickly off the recorded path: at approximately 3 m, **lane agreement** (the F1 score between lanes detected by CLRNNet on the render and on the real image) drops from 0.98 to 0.07.
- NVIDIA's released Difix improves image similarity (LPIPS 0.52 $\rightarrow$ 0.44) but not **lane-marking precision** (the fraction of painted-marking pixels in the render that match real markings; 0.52 $\rightarrow$ 0.48).
- Our re-implemented Difix, trained on neighbouring-lane pairs, raised lane agreement to 0.71 but produced more **spurious vehicles** (vehicles detected in the render with no counterpart in the real image: 0.18 per image, versus 0.04 for the unrepared render).

Open questions: whether these patterns hold across more roads and camera heights, how the smaller fixer and the road prior compare with Difix, and whether fixers trained on one dataset help on others. We expect mixed, scene-dependent results.

6. Evaluation Plan and Expected Results

- **Deviation axes:** lateral offset (real images from adjacent lanes at $\approx$ 3 m and 6–7.7 m); camera height (real car and scooter views from MV2; fallback: the real image re-projected onto the road plane at the new height); and heading (secondary: a pure rotation lets the real image be warped exactly to the new view, so only pixels the original camera saw are scored).
- **Metrics:** image similarity to the real image (PSNR and LPIPS, standard pixel and perceptual scores); lane-marking precision and recall (invented versus erased markings); lane-detector (CLRNNet [14]) and vehicle-detector (YOLO11) agreement, following the metrics of ReconDreamer [5]; spurious-vehicle count; and, as an extension, closed-loop evaluation, in which a released driving policy (LTF) drives inside the HUGSIM simulator [3] and is scored on safety and progress.
- **Expected results** (all methods of §4 and the raw render): *Fig. 1*, trust vs. lateral offset; *Fig. 2*, trust vs. camera height; *Table 1*, hallucination statistics; *Table 2*, cross-dataset transfer; *Table 3*, our Difix vs. the official code; side-by-side videos with hallucinations annotated.

Components and how each is used:

Component (what it does)	Source	Our use
Difix image fixer [1]: a diffusion model that takes a flawed rendered image and outputs a cleaned-up version	NVIDIA (code, weights)	Re-implemented (our training code) and trained on our pairs
Progressive 3D update [1]: cleaned images are added back as training views, so the 3D scene itself improves off the recorded path	Difix3D+ paper	Re-implemented
FaithFusion [2]: limits which pixels a fixer may change, using a per-pixel confidence map	Released code	Comparison method (run as released)
NVIDIA Fixer [4]: NVIDIA's Difix trained on driving data	Released weights	Comparison method (run as released)
Smaller fixer: standard image-to-image U-Net (15M parameters)	Standard architecture	Trained by us from scratch
Road-surface prior [15]: fits a smooth road surface to LiDAR points and flattens the road in the 3D scene; involves no image generation	ViSE paper (no code)	Re-implemented
3D reconstruction [13]: builds each splat scene from video	gsplat library	Run by us per scene
Detectors and simulator: CLRNNet lanes [14], YOLO11 vehicles, HUGSIM [3] with the LTF policy	Released, pretrained	Used as-is (evaluation only)

7. Timeline

Dates	Kyle Sha	Allen Xia
9/29–10/12	MV2 reconstruction and camera-height evaluation; additional Para-Lane test scenes	Evaluation framework for MV2 and EUVS; per-scene variance analysis
10/13–10/29	Validation of the Difix re-implementation against the official code	Road-surface prior on all scenes; hallucination analysis; mid-term report 10/29
10/30–11/20	Cross-dataset transfer study; progressive update on all scenes	Closed-loop evaluation in HUGSIM (extension)
11/21–12/10	Final figures and videos	Presentation (12/1–12/10) and project webpage (12/10)

Risks: preliminary results rest on few test roads (more scenes and per-scene variance will be reported); closed-loop tests use only LTF, as other policies need outdated dependencies.

References

1. Difix3D+, CVPR 2025 · arXiv 2503.01774	9. MV2 dataset, ECCV 2026 · arXiv 2608.12442
2. FaithFusion, CVPR 2026 · arXiv 2511.21113	10. Concept to Capability, SafeComp 2026 · arXiv 2605.01995
3. HUGSIM, TPAMI 2026 · arXiv 2412.01718	11. SetDiff, 2026 · arXiv 2601.07540
4. NVIDIA Fixer, 2025 · github.com/nv-tlabs/Fixer	12. ReCamDriving, ECCV 2026 · arXiv 2512.03621
5. ReconDreamer, CVPR 2025 · arXiv 2411.19548	13. 3D Gaussian Splatting, SIGGRAPH 2023 · arXiv 2308.04079
6. PRG-Fusion, 2026 · arXiv 2609.06948	14. CLRNNet, CVPR 2022 · arXiv 2203.10350
7. EUVS benchmark, ICCV 2025 · arXiv 2412.05256	15. ViSE, ICCV 2025 · arXiv 2510.18341
8. Para-Lane dataset, 3DV 2025 · arXiv 2502.15635	